☑ BACKEND REQUIREMENTS DOCUMENT FOR "VOICE BRIDGE"

📌 1. Project Overview (Backend Understanding)

Voice Bridge is a **real-time multilingual speech-to-speech translator** that performs:

- Live automatic speech recognition (ASR)
- Real-time text translation (NMT)
- Voice cloning-based speech synthesis (TTS)
- Multi-user meeting rooms
- Bluetooth and WebRTC-based audio streaming
- Transcript logging, meeting history & user identity management

Frontend is built in **React Native,** backend must support **real-time low-latency translation for English–Urdu–Arabic**.

📌 2. Backend Tech Stack (Required)

✓ Mandatory

- **Node.js**
- **Express.js** (REST API)
- **Socket.io** (WebSocket real-time signaling)
- **WebRTC signaling server**
- **MongoDB Atlas** (primary DB)
- **Firebase Authentication** OR JWT-based auth
- **Cloud Hosting:** Render / AWS / Vercel
- **AI APIs:**
  - Google Speech-to-Text
  - Google Translate
  - Resemble AI (for voice cloning + TTS)

✓ Optional Enhancements

- Whisper ASR (cloud deployed)
- NLLB-200 (offline server model)

📌 3. Backend Responsibilities

Your backend developer is responsible for these:

A) AUTHENTICATION

- Email/Password login
- Firebase Auth OR JWT tokens
- Secure user session handling
- Store user preferences (language, voice clone ID)

B) REAL-TIME AUDIO PIPELINE

Backend must receive continuous audio frames from frontend (WebRTC or Socket.io):

1 Incoming Audio Processing

- Receive PCM audio chunks
- Forward to ASR engine (Google Speech-to-Text streaming API)

2 Speech-to-Text (STT)

Backend sends audio to Google Speech-to-Text:

- Continuous recognition
- Returns word-by-word or sentence-by-sentence
- Supports English/Urdu/Arabic

3 Translation (NMT)

Backend sends STT result to:

- Google Translate API OR
- Meta NLLB (if implemented on server)

4 Text-to-Speech (TTS) with Voice Cloning

Backend sends translated text + speaker voice ID → Resemble AI
Backend returns audio buffer to frontend.

C) MULTI-USER MEETING ROOMS

Backend must create & manage:

- Create meeting room
- Join room
- Assign unique room IDs
- Real-time audio streams

- Real-time translated outputs to each participant
- Maintain participant list
- Handle WebRTC signaling (ICE, STUN/TURN)

Backend must support 3–5 participants per room with low latency.

D) TRANSCRIPT MANAGEMENT

Backend must:

- Store original speech text
- Store translated text
- Store timestamps
- Store speaker ID
- Store meeting ID

All stored in MongoDB.

E) BLUETOOTH AUDIO SUPPORT

Frontend handles Bluetooth routing.
Backend must support:

- Receiving audio frames from Bluetooth microphone
- Streaming translated output back to Bluetooth device

📌 4. Required API Endpoints

Your backend dev must build the following endpoints:

AUTH API

Method	Endpoint	Description
POST	/auth/register	Register user
POST	/auth/login	Login user (JWT or Firebase)
GET	/auth/me	Get current user profile
PUT	/auth/update	Update language/voice settings

MEETING API

Method	Endpoint	Description

POST	/meeting/create	Create new meeting room
POST	/meeting/join	Join existing room
GET	/meeting/:id	Get meeting data
GET	/meeting/:id/transcript	Fetch full meeting transcript

REAL-TIME STREAMING (SOCKET.IO)

Socket Events to Implement

Event Name	Purpose
user-joined	Notify participants user joined
user-left	Notify participants user left
audio-stream	Send raw audio chunks
stt-stream	Return speech-to-text chunks
translation-stream	Return translated text
tts-stream	Return generated speech
transcript-update	Live transcript logging

📌 5. Backend Data Models (MongoDB Schema)

User Schema

```
User {
  _id
  name
  email
  passwordHash (if JWT)
  preferredLanguage
  voiceCloneId (Resemble AI)
  createdAt
}
```

Meeting Schema

```
Meeting {
  _id
  createdBy
  participants: [userId]
  startTime
  endTime
}
```

Transcript Schema

```
Transcript {
  _id
  meetingId
  speakerId
  originalText
  translatedText
  timestamp
}
```

Voice Profile Schema

```
VoiceProfile {
  _id
  userId
  resembleVoiceId
  createdAt
}
```

📌 6. BACKEND SYSTEM WORKFLOW (Exact Pipeline)

Below steps must be implemented exactly:

1. Receive Audio

Frontend streams audio → backend (WebRTC or Socket.io)

2. ASR (Speech-to-Text)

Backend streams audio chunks → Google STT
Receives real-time text.

3. Translation

Backend sends text → Google Translate
Receives translated text.

4. Voice Cloning + TTS

Backend sends translated text + speaker voice ID → Resemble AI
Receives cloned voice audio.

5. Broadcast to Participants

Backend ensures:

- Each participant receives translated audio
- Each participant receives correct language version
- Uses WebRTC/Sockets

6. Transcript Logging

For every message:

- Save original text
- Save translation
- Save timestamps
- Save user + meeting ID

📌 7. Required Real-Time Architecture

Backend must implement:

✓ WebRTC Signaling

- Create SDP offers/answers
- Handle ICE candidates
- Use STUN/TURN server

✓ Socket.io Channels

For signaling + messaging.

✓ Streaming Buffers

Convert:

- PCM → Base64 → Stream
- STT output chunks
- TTS audio buffers

✓ **SRTP Encryption**

Handle secure transport.

📌 8. Backend Deployment Requirements

✓ **Upload backend to Render/Vercel/AWS**
✓ **Provide:**

- Base API URL
- WebSocket URL
- STUN/TURN server credentials
- .env file with configuration
- Postman API documentation
- Swagger API Docs (optional)

📌 9. Frontend Integration Expectations

Backend must return:

✓ **JSON responses**
✓ **Timestamps**
✓ **Speaker identification tags**
✓ **WebRTC room tokens**
✓ **Stream events for:**

- STT data
- Translation data
- TTS audio

Frontend will consume this using:

- React Native WebRTC
- React Native Sound
- Socket.io client

📌 10. Deliverables Required From Backend Developer

Your backend developer must deliver:

Documents

✓ API Documentation (Postman collection)
✓ Swagger/OpenAPI documentation
✓ README with environment setup
✓ System architecture diagram
✓ Database schema diagram
✓ WebRTC signaling guide

Backend Code

✓ Node.js codebase
✓ Socket.io + WebRTC server
✓ REST API
✓ Deployment link
✓ .env.example file

Live System

✓ Working backend URL
✓ Working WebSockets URL
✓ Working STUN/TURN servers

📌 11. Final Notes for Backend Developer

Your backend MUST be:

- **Low latency (< 2 seconds end-to-end)**
- **Scalable (support multiple rooms)**
- **Modular (ASR, NMT, TTS separate services)**
- **Secure (HTTPS, SRTP, Firebase/JWT)**
- **Cloud optimized**

This backend will be the engine powering the entire app's real-time multilingual communication.